Python for AI & Machine Learning

Class 12 – Instance Variables, Class Variables, and Types of Methods

Trainer: Surya Dekshith Mallikarjuna
Course: Python for AI & Machine Learning
Duration: 2 Hours (Theory + Practical)

Learning Objectives

By the end of this session, students will be able to:

- Understand the concept of Instance Variables.
- Understand the concept of Class Variables.
- Differentiate between Instance Variables and Class Variables.
- Learn the three types of methods in Python:
- Instance Methods
- Class Methods
- Static Methods
- Apply these concepts to real-world and AI & Machine Learning examples.

Revision of Previous Class

In the previous session, we learned:

- Introduction to Object-Oriented Programming (OOP)
- Classes
- Objects
- Attributes
- Methods
- The `self` keyword
- Constructors (`__init__()`)

Today, we will continue by learning how data and methods are managed inside a class.

Variables in a Class

Variables defined in a class are used to store data.

Python provides two main types of variables:

1. Instance Variables
2. Class Variables

Understanding these variables helps us design organized and reusable programs.

Instance Variables

An **Instance Variable** belongs to an individual object.

Every object created from a class has its own copy of the instance variables. Changing the value of an instance variable in one object does not affect any other object.

Characteristics

- Created using the `self` keyword.
- Defined inside the constructor (`__init__()`).
- Stores object-specific information.
- Each object has independent values.

Example

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Rahul", 22)
student2 = Student("Priya", 21)

print(student1.name)
print(student2.name)
```

Output

```
Rahul
Priya
```

Modifying an Instance Variable

Each object maintains its own data.

```python
class Student:

    def __init__(self, name):
        self.name = name

student = Student("Rahul")

print(student.name)

student.name = "Amit"

print(student.name)
```

Output

```
Rahul
Amit
```

Only the `student` object changes.

Class Variables

A **Class Variable** belongs to the class itself rather than to individual objects.

All objects share the same class variable.

Characteristics

- Declared directly inside the class.
- Shared by all objects.
- Stored only once in memory.
- Common information for all objects.

Example

```python
class Student:

    college = "Digix Academy"

    def __init__(self, name):
        self.name = name

student1 = Student("Rahul")
student2 = Student("Priya")
```

```
print(student1.college)
print(student2.college)
```

Output

```
Digix Academy
Digix Academy
```

Changing a Class Variable

A class variable can be modified using the class name.

```
class Student:

    college = "Digix Academy"

    def __init__(self, name):
        self.name = name

Student.college = "AI Academy"

student1 = Student("Rahul")
student2 = Student("Priya")

print(student1.college)
print(student2.college)
```

Output

```
AI Academy
AI Academy
```

All objects now reflect the updated value.

Instance Variables vs Class Variables

Instance Variables	Class Variables
Belong to each object	Belong to the class
Created using `self`	Declared directly in the class
Separate copy for each object	One shared copy

Instance Variables	Class Variables
Can have different values	Same value for all objects
Used for object-specific data	Used for common data

Methods in Python

Methods are functions defined inside a class.

Python provides three types of methods:

1. Instance Methods
2. Class Methods
3. Static Methods

Instance Methods

An **Instance Method** works with the data of an individual object.

It always receives the `self` parameter.

Characteristics

- Uses `self`.
- Can access and modify instance variables.
- Called using an object.

Example

```
class Student:

    def __init__(self, name):
        self.name = name

    def display(self):
        print("Student Name:", self.name)

student = Student("Rahul")

student.display()
```

Output

```
Student Name: Rahul
```

Class Methods

A **Class Method** works with class variables instead of instance variables.

It is created using the `@classmethod` decorator.

The first parameter is `cls`.

Characteristics

- Uses `cls`.
- Accesses class variables.
- Called using the class name or an object.

Example

```
class Student:

    college = "Digix Academy"

    @classmethod
    def display_college(cls):
        print("College:", cls.college)

Student.display_college()
```

Output

```
College: Digix Academy
```

Modifying a Class Variable Using a Class Method

```
class Student:

    college = "Digix Academy"

    @classmethod
    def change_college(cls, new_name):
```

```
        cls.college = new_name

Student.change_college("AI Academy")

print(Student.college)
```

Output

```
AI Academy
```

Static Methods

A **Static Method** does not depend on either object data or class data.

It is used for utility functions that logically belong to the class.

Static methods are created using the `@staticmethod` decorator.

Characteristics

- Does not use `self`.
- Does not use `cls`.
- Performs general-purpose operations.

Example

```
class Calculator:

    @staticmethod
    def add(a, b):
        return a + b

print(Calculator.add(10, 20))
```

Output

```
30
```

AI Example – Machine Learning Model

```
class MLModel:
```

```
    framework = "Scikit-learn"

    def __init__(self, model_name):
        self.model_name = model_name

    def train(self):
        print(self.model_name, "Model Training Started")

    @classmethod
    def display_framework(cls):
        print("Framework:", cls.framework)

    @staticmethod
    def version():
        print("Machine Learning Framework Version 1.0")

model = MLModel("Linear Regression")

model.train()
MLModel.display_framework()
MLModel.version()
```

AI Example – Student Management System

```
class AIStudent:

    institute = "Digix AI Academy"

    def __init__(self, name, course):
        self.name = name
        self.course = course

    def display(self):
        print("Name:", self.name)
        print("Course:", self.course)

    @classmethod
    def institute_name(cls):
        print("Institute:", cls.institute)

    @staticmethod
    def welcome():
        print("Welcome to AI & Machine Learning")

student = AIStudent("Surya", "Python for AI")

student.display()
```

```
AIStudent.institute_name()
AIStudent.welcome()
```

Real-World Example – Employee Management

```
class Employee:

    company = "Digix Technologies"

    def __init__(self, name, department):
        self.name = name
        self.department = department

    def display(self):
        print(self.name, "-", self.department)

    @classmethod
    def company_name(cls):
        print("Company:", cls.company)

    @staticmethod
    def greeting():
        print("Welcome to Digix Technologies")

employee = Employee("Rahul", "AI")

employee.display()
Employee.company_name()
Employee.greeting()
```

Advantages of Instance Variables

- Store object-specific information.
- Each object has independent data.
- Easy to modify individual object values.
- Makes programs flexible.

Advantages of Class Variables

- Saves memory.
- Common data is stored only once.
- Easy to update shared information.
- Suitable for constants and common settings.

Advantages of Static Methods

- Ideal for utility functions.
- Improve code organization.
- Do not require object creation.
- Can be called directly using the class name.

Summary

In today's class, we learned:

- Instance Variables
- Class Variables
- Differences between Instance Variables and Class Variables
- Instance Methods
- Class Methods
- Static Methods
- AI and real-world implementation examples

These concepts are essential because almost every Python framework and AI library uses classes that contain instance variables, class variables, and different types of methods.

Practical Exercises

Practical 1 – Car Class

Create a `Car` class with:

- Brand
- Model

Class Variable:

- Company

Methods:

- Display Car Details
- Display Company Name
- Welcome Message

Create two car objects.

Practical 2 – Bank Account

Create a `BankAccount` class.

Include:

- Account Number
- Customer Name
- Balance

Class Variable:

- Bank Name

Methods:

- Display Account Details
- Change Bank Name
- Bank Information

Practical 3 – AI Model

Create an `AIModel` class.

Store:

- Model Name
- Accuracy

Class Variable:

- Framework

Methods:

- Display Model Information
- Change Framework
- Display Framework
- About the Framework

Assignment

Develop a **Library Management System** using Object-Oriented Programming.

Create a `Book` class with:

Instance Variables

- Book ID
- Book Name
- Author
- Price

Class Variable

- Library Name

Methods

- Display Book Details (Instance Method)
- Change Library Name (Class Method)
- Display Library Rules (Static Method)

Create details for **five books** and demonstrate all three types of methods.

Next Class Preview

In the next session, we will begin the **Four Pillars of Object-Oriented Programming**, which are the foundation of modern software development:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

These concepts are widely used in AI frameworks, web applications, enterprise software, and large-scale Python projects.